Creating an E-Commerce System with SQL – Detailed Notes

Introduction

An **E-Commerce System** is an application that allows customers to browse products, add them to a cart, place orders, and make payments online.

SQL is used to store and manage all the information related to:

- Customers
- Products
- Categories
- Orders
- Payments
- Shopping Cart
- Reviews

Step 1: Database Design

Before writing SQL queries, we need to identify the entities involved.

Main Entities

Entity	Purpose
Customers	Stores customer details
Categories	Product categories
Products	Information about products
Cart	Temporary shopping cart
Orders	Customer orders
Order Items	Products inside an order
Payments	Payment details
Reviews	Customer reviews

Step 2: Creating the Database

```
CREATE DATABASE ecommerce_db;

USE ecommerce_db;
```

Step 3: Creating Tables

---

1. Customers Table

Stores customer information.

```
CREATE TABLE Customers(

customer_id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(100),

email VARCHAR(100) UNIQUE,

phone VARCHAR(15),

address TEXT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

Explanation

Column	Description
customer_id	Unique customer ID
name	Customer Name
email	Email Address
phone	Contact Number
address	Delivery Address
created_at	Registration date

Insert Sample Data

```
INSERT INTO Customers(name,email,phone,address)

VALUES

('John Doe',
'john@gmail.com',
'9876543210',
'New York'),
```

('Jane Smith',
'jane@gmail.com',
'9988776655',
'California');

---

2. Categories Table

Products belong to categories.

CREATE TABLE Categories(

category_id INT PRIMARY KEY AUTO_INCREMENT,

category_name VARCHAR(100)

);

Insert Categories

INSERT INTO Categories(category_name)

VALUES

('Electronics'),

('Fashion'),

('Books');

---

3. Products Table

CREATE TABLE Products(

product_id INT PRIMARY KEY AUTO_INCREMENT,

product_name VARCHAR(150),

description TEXT,

price DECIMAL(10,2),

stock_quantity INT,

category_id INT,

FOREIGN KEY(category_id)

REFERENCES Categories(category_id)

);

Insert Products

INSERT INTO Products

(product_name,description,price,stock_quantity,category_id)

VALUES

('Laptop',
'Gaming Laptop',
75000,
15,
1),

('T-shirt',
'Cotton T-shirt',
800,
50,
2);

Viewing Products

SELECT * FROM Products;

---

4. Shopping Cart Table

CREATE TABLE Cart(

cart_id INT PRIMARY KEY AUTO_INCREMENT,

customer_id INT,

product_id INT,

quantity INT,

FOREIGN KEY(customer_id)

```
REFERENCES Customers(customer_id),

FOREIGN KEY(product_id)

REFERENCES Products(product_id)

);
```

Add Products to Cart

```
INSERT INTO Cart(customer_id,product_id,quantity)

VALUES

(1,1,2),

(1,2,1);
```

Display Cart Details

```
SELECT


Customers.name,

Products.product_name,

Cart.quantity,

Products.price,

(Cart.quantity * Products.price)

AS Total


FROM Cart


JOIN Customers

ON Cart.customer_id = Customers.customer_id


JOIN Products
```

ON Cart.product_id = Products.product_id;

---

5. Orders Table

Stores basic order information.

CREATE TABLE Orders(

order_id INT PRIMARY KEY AUTO_INCREMENT,

customer_id INT,

order_date TIMESTAMP

DEFAULT CURRENT_TIMESTAMP,

status VARCHAR(50)

DEFAULT 'Pending',

FOREIGN KEY(customer_id)

REFERENCES Customers(customer_id)

);

Insert Order

INSERT INTO Orders(customer_id)

VALUES(1);

---

6. Order Items Table

Each order may contain multiple products.

CREATE TABLE Order_Items(

order_item_id INT PRIMARY KEY AUTO_INCREMENT,

```
order_id INT,

product_id INT,

quantity INT,

price DECIMAL(10,2),

FOREIGN KEY(order_id)

REFERENCES Orders(order_id),

FOREIGN KEY(product_id)

REFERENCES Products(product_id)

);
```

Insert Ordered Products

```
INSERT INTO Order_Items

(order_id,product_id,quantity,price)

VALUES

(1,1,2,75000),

(1,2,1,800);
```

View Order Details

```
SELECT

Orders.order_id,

Customers.name,
```

Products.product_name,

Order_Items.quantity,

Order_Items.price,

(Order_Items.quantity *

Order_Items.price)

AS Amount

FROM Order_Items

JOIN Orders

ON Orders.order_id = Order_Items.order_id

JOIN Customers

ON Customers.customer_id = Orders.customer_id

JOIN Products

ON Products.product_id = Order_Items.product_id;

---

7. Payments Table

CREATE TABLE Payments(

payment_id INT PRIMARY KEY AUTO_INCREMENT,

order_id INT,

payment_method VARCHAR(50),

amount DECIMAL(10,2),

payment_status VARCHAR(50),

payment_date TIMESTAMP

DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY(order_id)

REFERENCES Orders(order_id)

);

Insert Payment

INSERT INTO Payments

(order_id,payment_method,amount,payment_status)

VALUES

(1,

'UPI',

150800,

'Completed');

---

8. Reviews Table

Customers can review purchased products.

CREATE TABLE Reviews(

review_id INT PRIMARY KEY AUTO_INCREMENT,

customer_id INT,

product_id INT,

rating INT,

comment TEXT,

FOREIGN KEY(customer_id)

REFERENCES Customers(customer_id),

FOREIGN KEY(product_id)

REFERENCES Products(product_id)

);

Insert Review

INSERT INTO Reviews

(customer_id,

product_id,

rating,

comment)

VALUES

(1,

1,

5,

'Excellent Laptop');

Step 4: Important SQL Queries for E-Commerce System

---

1. Find All Products

```
SELECT * FROM Products;
```

---

2. Search Product

```
SELECT *

FROM Products

WHERE product_name

LIKE '%Laptop%';
```

---

3. Products with Category Name

```
SELECT

product_name,

category_name

FROM Products

JOIN Categories

ON Products.category_id

= Categories.category_id;
```

---

4. Total Amount of an Order

```
SELECT

SUM(quantity*price)

AS Total_Amount

FROM Order_Items

WHERE order_id=1;
```

---

5. Available Products

```
SELECT *

FROM Products

WHERE stock_quantity > 0;
```

---

6. Top Selling Products

```
SELECT

Products.product_name,

SUM(Order_Items.quantity)

AS TotalSold

FROM Order_Items

JOIN Products

ON Products.product_id
```

= Order_Items.product_id

GROUP BY product_name

ORDER BY TotalSold DESC;

---

7. Customer Order History

SELECT

Customers.name,

Orders.order_id,

Orders.status,

Orders.order_date

FROM Orders

JOIN Customers

ON Customers.customer_id

= Orders.customer_id

WHERE Customers.customer_id = 1;

Step 5: Updating Stock Automatically

When an order is placed, product quantity should decrease.

Example

```
UPDATE Products

SET stock_quantity = stock_quantity - 2

WHERE product_id = 1;
```

Step 6: Deleting Cart Items After Order Placement

```
DELETE FROM Cart

WHERE customer_id = 1;
```

Step 7: Using Transactions

Transactions ensure that all operations succeed together.

```
START TRANSACTION;

INSERT INTO Orders(customer_id)

VALUES(1);

UPDATE Products

SET stock_quantity = stock_quantity-1

WHERE product_id=1;

INSERT INTO Payments(

order_id,

payment_method,

amount,

payment_status

)

VALUES
```

(1,

'Credit Card',

75000,

'Completed');

COMMIT;

If something goes wrong:

ROLLBACK;

Step 8: ER Diagram Structure

Customers
  |
  |
Orders
  |
Order_Items
  |
Products
  |
Categories

Customers
  |
Cart
  |
Products

Orders
  |
Payments

Customers
  |
Reviews
  |
Products

Best Practices for an E-Commerce Database

Use Constraints

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK Constraints

Index Frequently Searched Columns

```
CREATE INDEX idx_product_name

ON Products(product_name);
```

Store Passwords Securely

Never store plain-text passwords.

Use hashed passwords such as:

- BCrypt
- Argon2
- SHA-256 (with salt)

Normalize Data

- Avoid duplicate data.
- Separate products, categories, and orders into different tables.

Features That Can Be Added Later

- Wishlist Management
- Coupon System
- Product Images
- Inventory Tracking
- Shipping Addresses
- Multiple Payment Gateways
- Order Cancellation and Refunds

- Admin Dashboard
- Vendor/Seller Management
- Product Recommendations
- GST and Invoice Generation

These tables and queries together form a complete SQL-based E-Commerce Management System suitable for beginner to intermediate SQL projects and CRUD operation demonstrations.